



Workshop java profiling

Victor Gallet
Fabrice Bibonne

2026



Objectifs de ce workshop

- utiliser async-profiler
- utiliser JDK Flight Recorder (JFR) et JDK Mission Control (JMC)
- Utiliser le meilleur outil de performance testing : k6
- Apprenez à profiler votre application Java
- Apprenez à lire et interprétez un Flamegraph
- N'ayez plus peur des problèmes de performance!

Resources

- [async-profiler](#)
- [JFR et JMC](#)
- [jvmpref](#)
- [Coloring Flame Graphs: Code Hues](#) by Brendan Gregg
- [A Guide to async-profiler](#) by Anshul Bansal and Eric Martin
- [JDK Mission Control tutorial](#) by Marcus Hirt
- [FUSENIX ATC '17: Visualizing Performance with Flame Graphs](#) by Brendan Gregg
- [Taming performance issues into the wild: a practical guide to JVM profiling](#) by Francesco Nigro, Mario Fusco
- [\[Java\]\[Profiling\] Async-profiler - manual by use cases](#) by Krzysztof Ślusarski
- [\[Java\]\[Profiling\]\[Memory leak\] Finding heap memory leaks with Async-profiler](#) by Krzysztof Ślusarski
- [Java Safepoint and Async Profiling](#) by Seetha Wenner
- [FRTraquer une fuite mémoire : cas d'étude avec Hibernate 5, ne tombez pas dans le IN](#) by Ling-Chun SO
- [FESous le capot d'une application JVM - Java Flight Recorder / Java Mission Control](#)
- [by Guillaume Darmont](#)
- [FPerformance et diagnostic - Méthodologie et outils](#) by Vladislav Pernin



async-profiler késako ?

- low overhead sampling profiler
- se base sur une API non standard de Hotspot JVM ([issue AsyncGetCallTrace replacement](#))
- ne souffre pas du [Safepoint biais problem](#)
- permet de profiler plusieurs types d'événements :
 - CPU
 - Memory Allocation
 - Wall
 - Lock
 - Java Method Profiling



Utiliser async-profiler

- En process standalone

```
> asprof -d <duration> -e <event> -f /tmp/flamegraph.html <pid>
```

- En tant qu'agent

```
> java
```

```
-agentpath:/path/to/libasyncProfiler.so=start,event=cpu,file=profile.html ...
```



Utiliser async-profiler

- Dans un container

```
> wget https://github.com/async-profiler/async-profiler/releases/download/v3.0/async-profiler-3.0-linux-x64.tar.gz
```

```
> tar -xzf async-profiler-3.0-linux-x64.tar.gz
```

```
> asprof -d <duration> -e <event> -f /tmp/flamegraph.html <pid>
```

<https://github.com/async-profiler/async-profiler/blob/master/docs/ProfilingInContainer.md>

JFR késako ?

- **JDK Flight Recorder**
- **low** overhead **event-based observability** and monitoring framework **built into** the HotSpot JVM
- JEP 167 - Event-Based JVM Tracing (java 7) puis JEP 328 - Flight Recorder (java 11) puis...
- Événements émis vers la mémoire, un fichier, vers un client JMX, ...
- Se débat parfois avec le [Safepoint biais problem](#) => JEP 518 (JFR Cooperative Sampling)
- permet de déboguer et profiler les applications
- permet de profiler plusieurs types d'[événements](#) :
 - Threads, CPU
 - Mémoire et allocations
 - IO, exceptions, verrous
 - GC, compilations, class loading
 - Environnements
 - Personnalisés



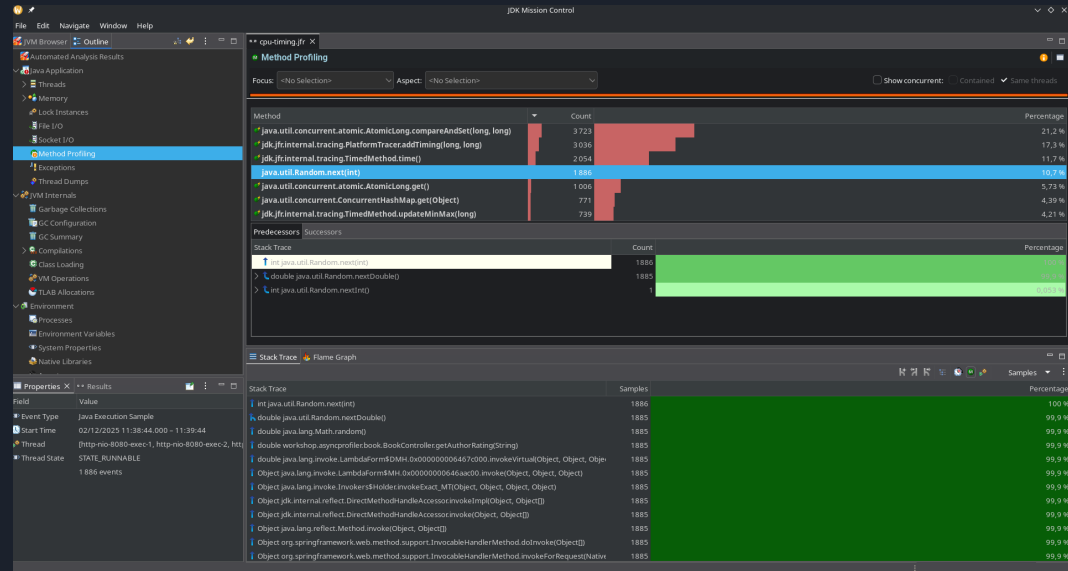


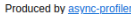
Utiliser JFR

- Démarrer l'enregistrement :
 - Lancement au démarrage de la JVM : `-XX:StartFlightRecording ...`
 - Lancement pour une JVM déjà démarrée :
`jcmd $PID JFR.start ... ; jcmd $PID JFR.configure ...`
- Dump d'un enregistrement dans un fichier : `jcmd $PID JFR.dump`
- Arrêt d'un enregistrement : `jcmd $PID JFR.stop name=1`
- Utilisation depuis l'application via [l'API JFR du JDK](#) => JFR Unit ...
- Possibilité de commande à distance *également*
- Et pourquoi pas dans un contexte d'observabilité ?

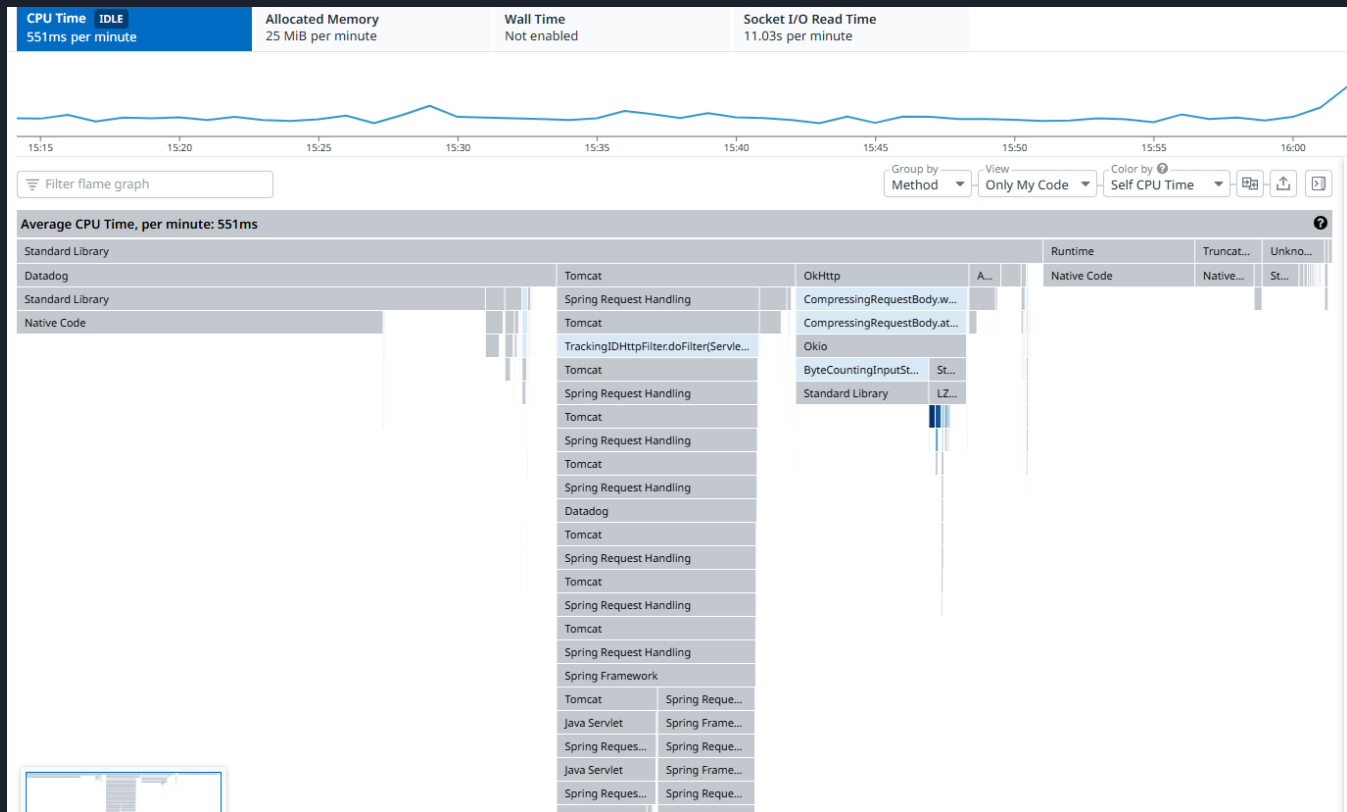
Lire les résultats d'un enregistrement JFR => JMC / jfr

- JMC : JDK Mission Control
 - Application de bureau pour visualiser les résultats d'un enregistrement
 - Navigateur d'évènements
 - Vues thématiques (Threads, Mémoire, Hot methods, ...)
 - Vues personnalisables
 - Analyses automatiques
- jfr (outil en console)
 - Afficheur d'évènements
 - Manipulation d'enregistrements
 - Vues thématiques (hot methods, gc, allocations...)
- API [jdk.jfr.consumer](https://docs.oracle.com/javase/10/other/jdk-jfr/jdk-jfr-consumer.html)

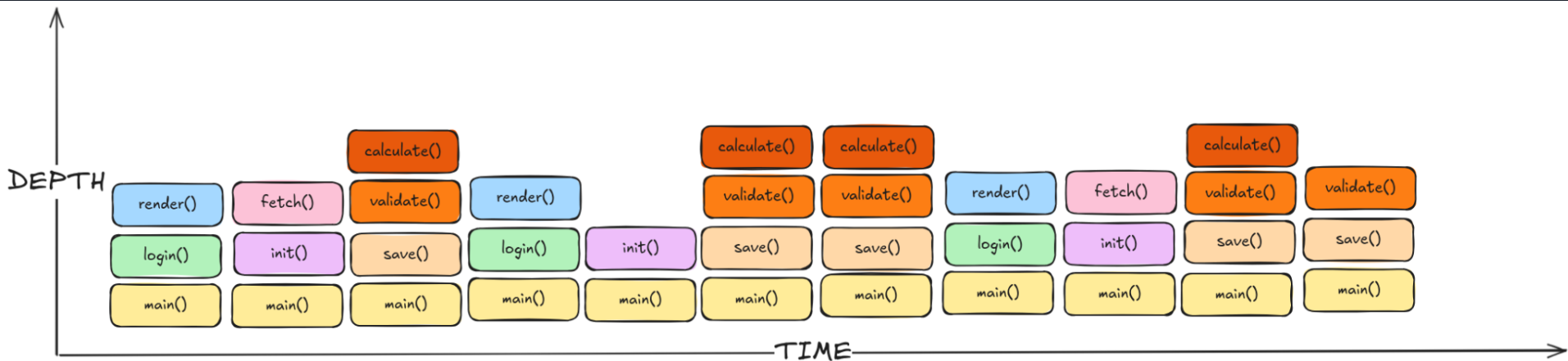




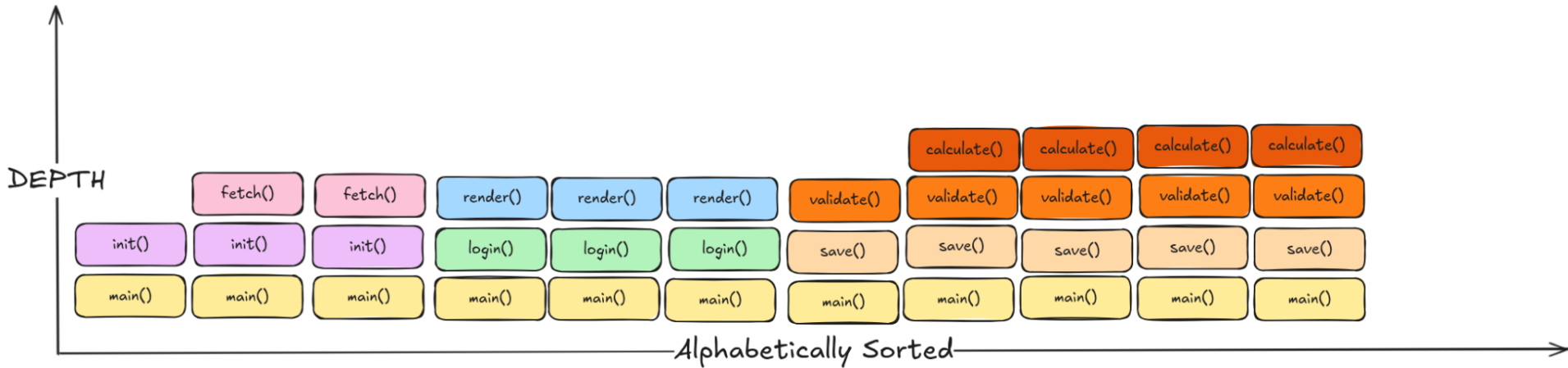
Interpréter un Flamegraph



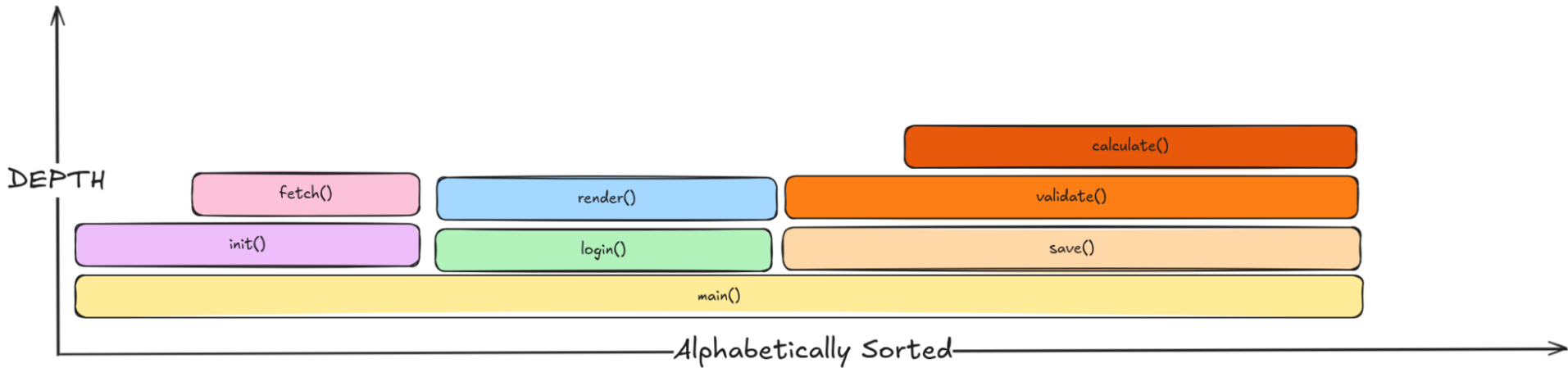
Construction d'un Flamegraph



Tri par ordre alphabétique



Agrégation



la méthode `calculate()` est un bon candidat à l'optimisation car elle est plus "large" sur l'axe du temps et de la profondeur dans la stack.

Les méthodes `render()` et `fetch()` peuvent être des candidats de second choix.

Code Couleur d'un Flamegraph *async-profiler*

Kernel	__x64_sys_futex
	do_syscall_64
	entry_SYSCALL_64_after_hwframe
Native	__lll_lock_wake
	pthread_mutex_unlock
C++	SharedRuntime::handle_ic_miss_helper_internal
	SharedRuntime::handle_ic_miss_helper
	SharedRuntime::handle_wrong_method_ic_miss
Native	ic_miss_stub
C1 compiled	java/util/ComparableTimSort.countRunAndMakeAscending
Interpreted	java/util/ComparableTimSort.sort
	java/util/Arrays.sort
Java inlined	java/util/Arrays.sort
Java compiled	java/util/ArrayList.sort
Match found	java/util/Collections.sort
	nonapi/io/github/classgraph/Utils/CollectionUtils.sortIfNotEmpty

- System (User native)
- Java
- C++



Injecter de la charge avec k6

- Outil open source de test de charge automatisés
- Scénarios en JS ou TS (qui peuvent être générés par IA ou autre)
- Définition de seuils et de critères pour vérifier le bon fonctionnement
- Emulateur de navigateur (avec convertisseur de scripts playwright)
- Résultats multiformes
- Nombreuses métriques personnalisables
- Multi plateformes : Linux, Mac, Windows, *via un container ...*



Durant ce workshop ?

- Nous allons profiler une application Java avec des exemples plus ou moins factices
- Merci de vous arrêter à chaque étape pour que nous prenions le temps d'expliquer
- Bien utiliser k6 avant chaque session de profiling !